

PMD · Project Roadmap Architecture

Artifact #18 · v1.0 · May 2026 · PMD System · Internal — Not for Public Distribution

ARCHITECTURE INTENT

PHASE OVERVIEW

Phase	Title	Projects	Core Capability Unlocked
I	FOUNDATION	P1 – P6	Static Web & Developer Fundamentals
II	FRONT-END FRAMEWORK	P7 – P13	React & Component Architecture
III	BACKEND & DATABASE	P14 – P20	Node, APIs, and Persistence
IV	PAYMENTS, GROWTH & ANALYTICS	P21 – P27	Monetization Layer
V	REAL-TIME & SCALE	P28 – P33	WebSockets, Queues, and Caching
VI	AI & MACHINE LEARNING INTEGRATION	P34 – P41	AI & MACHINE LEARNING INTEGRATION
VII	PLATFORM, MOBILE & PRODUCTION SCALE	P42 – P50+	PLATFORM, MOBILE & PRODUCTION SCALE

Phase I · P1 – P6

FOUNDATION — Static Web & Developer Fundamentals

Build the bedrock. Master HTML, CSS, JavaScript, Git, and deployment before touching any framework or library. Every professional developer returns to these fundamentals under pressure — own them first. PM skill introduced: problem framing, user story writing, and basic PRD drafting.

Project	Scope	PM Growth	Tech Growth	New Tools	Artifacts
	Personal Link-in-Bio Page				
P1	Single-page static site with profile section, curated links, and social icons. No interactivity beyond hover states.	Problem framing · Single-user persona · Basic PRD · Definition of done	HTML5 semantic structure · CSS Flexbox · CSS custom properties · Git init and first commit · Netlify deploy	HTML5 · CSS3 · Git · GitHub · Netlify	PRD v1 · Persona · Deployed URL
	■ First ship. Prove the CI/CD pipeline works end to end. Nothing else matters yet.				
	Responsive Marketing Landing Page				
P2	Multi-section landing page with hero, features, testimonials, and CTA. Fully responsive across mobile, tablet, desktop.	User journey mapping · Above-fold vs below-fold hierarchy · CTA copy strategy	CSS Grid · Responsive breakpoints · Mobile-first methodology · WCAG 2.2 AA · CSS animations	HTML5 · CSS3 Grid/Flexbox · Vanilla JS · axe-core (accessibility audit)	Wireframe · PRD · Accessibility audit report
	■ WCAG 2.2 AA is non-negotiable from this project forward. axe-core introduced here.				
	Interactive JavaScript Quiz App				
P3	Browser-based quiz with multiple question types, score tracking, progress bar, and results screen. State managed in memory — no backend.	User flow mapping · Acceptance criteria per user story · Edge case documentation	DOM manipulation · Event listeners · JS state management in memory · Conditional rendering · localStorage	Vanilla JS · localStorage · CSS animations	User flow diagram · PRD with acceptance criteria · Edge case log
	Task Manager with Persistent Local State				
P4	CRUD task board — create, read, update, delete tasks. Filter by status. Persist state across sessions via localStorage. Drag-to-reorder.	CRUD interaction design · Prioritization logic · Data model definition	localStorage read/write · JSON serialization · Drag-and-drop API · ARIA live regions	Vanilla JS · localStorage · HTML Drag and Drop API	Data model spec · PRD · Interaction spec
	■ First exposure to data modeling. How data is structured determines how the UI behaves.				
	Weather Dashboard with Public API				

Project	Scope	PM Growth	Tech Growth	New Tools	Artifacts
P5	Location-based weather dashboard consuming a public REST API. Current conditions, 5-day forecast, unit toggle (°F/°C). Error and loading states handled.	API contract documentation · Error state UX · Loading state design · Third-party dependency risk	Fetch API · Async/Await · REST API consumption · JSON parsing · Error handling patterns	OpenWeatherMap API · Fetch API · Async JS · Chart.js (forecast visualization)	API integration spec · Error state matrix · PRD
	Multi-Page Portfolio Site with Vanilla JS Routing				
P6	Rebuild thepmdev.com foundation as a multi-page site with client-side routing simulation, animated transitions, and dynamic content injection. No framework.	IA (information architecture) · Navigation design · Content hierarchy · SEO fundamentals	History API / hash routing · JS module pattern · CSS transitions · Meta tags · Open Graph tags	Vanilla JS modules · History API · CSS transitions · Google Analytics 4	IA diagram · Navigation spec · PRD
	■ Closes Phase I. Before touching React, understand what a framework is actually solving.				

Phase II · P7 – P13

FRONT-END FRAMEWORK — React & Component Architecture

Graduate from document-oriented thinking to component-oriented thinking. React is the dominant front-end framework in product companies. Master it deeply — components, state, hooks, routing, testing — before adding a backend. PM skill introduced: design systems, component spec writing, A/B test hypothesis framing.

Project	Scope	PM Growth	Tech Growth	New Tools	Artifacts
	React Component Library & Design System				
P7	Build a reusable component library: Button, Input, Card, Modal, Toast, Badge. Document each component with usage examples. No app logic — pure UI primitives.	Component spec writing · Design token documentation · Reusability criteria	React functional components · Props and prop types · CSS Modules · Storybook-style documentation	React · Vite · CSS Modules · TypeScript (introduced)	Component spec · Design token inventory · PRD
	Rebuilding the Task Manager in React				
P8	Port P4 Task Manager to React. Add tag filtering, priority levels, due dates, and a Kanban board view. State via useState and useReducer.	Feature parity analysis · Migration planning · Scope delta documentation	useState · useReducer · useEffect · Conditional rendering · Component composition	React · Vite · TypeScript · Tailwind CSS	Migration PRD · Feature delta log · Kanban interaction spec
	■ Migration projects are a core PM skill. Document what changed, what was preserved, and why.				
	Multi-Page SPA with React Router				
P9	Content-driven multi-page application — blog or knowledge base — with dynamic routing, nested routes, 404 handling, and breadcrumb navigation.	URL structure design · Deep-link strategy · Navigation UX patterns	React Router v6 · Dynamic route params · Nested layouts · Lazy loading routes	React Router · React · Vite · TypeScript	Route map · IA diagram · PRD
	Real-Time Data Dashboard with API Polling				
P10	Live financial or sports data dashboard. Multiple chart types. Auto-refresh on interval. Skeleton loading states. Data caching in memory.	Refresh rate UX tradeoffs · Data freshness SLA definition · Dashboard layout hierarchy	useEffect polling · Custom hooks · Chart.js or Recharts · Skeleton UI patterns · Memoization	React · Recharts · CoinGecko API or similar · TypeScript	Dashboard spec · Refresh rate decision doc · PRD
	Global State Management — Shopping Cart				
P11	E-commerce product listing and cart. Add/remove/update quantities. Persist cart to LocalStorage. Global state via Zustand or Redux Toolkit.	Cart UX patterns · State persistence strategy · Checkout flow design	Zustand or Redux Toolkit · Context API comparison · Selector patterns · Middleware	React · Zustand or Redux Toolkit · TypeScript · Tailwind CSS	State architecture diagram · Cart flow spec · PRD

Project	Scope	PM Growth	Tech Growth	New Tools	Artifacts
	Component Testing with Jest and React Testing Library				
P12	Retroactively add a full test suite to P8 or P11. Unit tests per component. Integration tests for key user flows. Coverage report generated.	Acceptance criteria as test cases · QA strategy documentation · Release confidence definition	Jest · React Testing Library · Test coverage tooling · Mocking · CI test runs	Jest · React Testing Library · GitHub Actions (CI)	Test strategy doc · Coverage report · PRD
	■ <i>Testing is a PM skill as much as a dev skill. Acceptance criteria should be directly traceable to tests.</i>				
	Next.js SSR/SSG Content Site with CMS				
P13	Content-driven site (case studies, blog, or documentation) powered by a headless CMS. Static generation for performance. Dynamic pages where needed.	Content modeling · SEO strategy · Static vs dynamic routing tradeoffs	Next.js · Static Site Generation · Server-Side Rendering · Contentful or Sanity · Image optimization	Next.js · Contentful or Sanity · TypeScript · Vercel	Content model spec · SEO checklist · PRD
	■ <i>Closes Phase II. Next.js is the most common React framework in production product companies.</i>				

Phase III · P14 – P20

BACKEND & DATABASE — Node, APIs, and Persistence

Cross the full-stack line. Build your own APIs, connect to real databases, and understand the server-side decisions that drive product tradeoffs. PM skill introduced: API contract design, data modeling, technical requirement writing, and backend system architecture documentation.

Project	Scope	PM Growth	Tech Growth	New Tools	Artifacts
	REST API with Node.js and Express				
P14	Build a standalone REST API for the P4/P8 Task Manager. CRUD endpoints. Request validation. Error handling middleware. Tested via Postman.	API contract documentation · Endpoint design · HTTP status code strategy · Versioning rationale	Node.js · Express.js · RESTful routing · Middleware · Postman · JSON API design	Node.js · Express.js · Postman · dotenv	API contract doc · Postman collection · PRD
	PostgreSQL Integration with Prisma ORM				
P15	Replace LocalStorage in P14 with a PostgreSQL database. Schema design, migrations, and relational queries via Prisma ORM.	Data model documentation · Migration planning · Relational vs flat data tradeoffs	PostgreSQL · Prisma ORM · Database migrations · Schema design · SQL fundamentals	PostgreSQL · Prisma · Node.js · Express.js	Data model diagram · Migration log · PRD
	■ SQL is a non-negotiable PM hard skill. Every data-fluent PM can write a basic SELECT query.				
	User Authentication — JWT and Session Management				
P16	Add authentication to the P14/P15 API. Email + password registration and login. JWT access tokens. Refresh token rotation. Protected routes.	Auth flow documentation · Security requirement writing · Session UX design	JWT · bcrypt · HTTP-only cookies · Refresh token pattern · Role-based route protection	JWT · bcrypt · Node.js · Express.js · PostgreSQL	Auth flow diagram · Security requirements doc · PRD
	Full-Stack App — React Frontend + Node API + PostgreSQL				
P17	First fully integrated full-stack application. React frontend consumes the authenticated Node/Express/PostgreSQL API. Users can register, log in, and manage their own data.	Full system architecture documentation · Cross-layer dependency mapping · End-to-end user journey	Full-stack integration · CORS · Environment variables · Deployment pipeline (frontend + backend)	React · Node.js · Express · PostgreSQL · Prisma · Render or Railway (backend hosting)	System architecture diagram · Full-stack PRD · Deployment runbook
	■ First project where every layer is yours. The debugging will be instructive.				
	Role-Based Access Control (RBAC) System				

Project	Scope	PM Growth	Tech Growth	New Tools	Artifacts
P18	Extend P17 with admin and standard user roles. Admin dashboard with user management. Permission-gated UI elements. Audit log of admin actions.	Permission matrix design · Admin UX patterns · Compliance-adjacent documentation	RBAC middleware · Permission-gated rendering · Audit logging · Admin UI patterns	Node.js · PostgreSQL · React · JWT with roles	Permission matrix · Admin flow spec · PRD
	NoSQL with MongoDB — Event or Activity Feed				
P19	Build an activity feed or event log system using MongoDB. Flexible document schema. Pagination. Filter and sort. Compare tradeoffs vs PostgreSQL.	SQL vs NoSQL tradeoff documentation · Schema flexibility cost-benefit analysis	MongoDB · Mongoose · Document schema design · Aggregation pipeline · Indexed queries	MongoDB · Mongoose · Node.js · Express.js	SQL vs NoSQL decision doc · Schema design · PRD
	Transactional Email System with SendGrid				
P20	Add transactional email to P17: welcome email on registration, password reset flow, notification emails. Email templates. Delivery tracking.	Transactional email spec writing · User communication design · Deliverability requirements	SendGrid API · Email templating · Webhook handling · Queue introduction	SendGrid · Node.js · Express.js · HTML email templates	Email flow spec · Template inventory · PRD
	■ Closes Phase III. Communication design is a PM craft — every automated message is a product decision.				

Phase IV · P21 – P27

PAYMENTS, GROWTH & ANALYTICS — Monetization Layer

Build products that make money and measure themselves. Stripe integration, subscription logic, feature flags, A/B testing, and behavioral analytics are the tools of growth-oriented PMs. PM skill introduced: pricing model design, funnel analysis, experiment design, and KPI instrumentation.

Project	Scope	PM Growth	Tech Growth	New Tools	Artifacts
	Stripe Payment Integration — One-Time Purchase				
P21	Add one-time payment processing to a product. Stripe Checkout or Elements. Webhook handling for payment confirmation. Purchase confirmation email.	Payment flow documentation · Checkout UX design · Failure state handling · PCI compliance awareness	Stripe API · Stripe Webhooks · Idempotency keys · Payment intent lifecycle	Stripe · Node.js · Express.js · React	Payment flow diagram · Webhook event spec · PRD
	Subscription Billing with Stripe — SaaS Model				
P22	Full subscription lifecycle: free trial, plan selection, upgrade/downgrade, cancellation, reactivation. Billing portal. Invoice history.	Pricing tier design · Subscription lifecycle mapping · Churn definition · LTV/CAC fundamentals	Stripe Subscriptions · Stripe Customer Portal · Billing webhooks · Plan entitlement logic	Stripe · Node.js · PostgreSQL · React	Pricing tier spec · Subscription lifecycle diagram · PRD
	■ <i>Subscription logic is the most common PM domain in SaaS. Own it completely.</i>				
	Feature Flag System with LaunchDarkly				
P23	Implement feature flags across P17 or P22. Gradual rollout, kill switch, per-user targeting, and percentage rollout. Flag inventory documented.	Rollout strategy design · Flag lifecycle management · Percentage rollout risk assessment	LaunchDarkly SDK · Flag targeting rules · A/B flag variants · Flag cleanup discipline	LaunchDarkly · React · Node.js	Flag inventory · Rollout plan · PRD
	Product Analytics with Mixpanel or Amplitude				
P24	Instrument a product with behavioral event tracking. Define event taxonomy. Build funnels, retention cohorts, and a feature adoption dashboard.	Event taxonomy design · Funnel definition · Retention analysis · AARRR framework application	Mixpanel or Amplitude SDK · Event tracking · Funnel visualization · Cohort analysis	Mixpanel or Amplitude · Segment.io (optional) · React	Event taxonomy doc · Funnel spec · AARRR dashboard · PRD
	■ <i>The event taxonomy you design here is the single most important data decision a PM makes.</i>				
	A/B Testing Framework — Hypothesis to Results				

Project	Scope	PM Growth	Tech Growth	New Tools	Artifacts
P25	Design and run a real A/B test on a live product feature. Hypothesis, sample size calculation, variant implementation, statistical significance analysis, and rollout decision.	A/B test design · Statistical significance · MDE calculation · Results interpretation · Rollout/rollback decision	LaunchDarkly variants · Chi-square or z-test · Sample size calculator · Results visualization	LaunchDarkly · Mixpanel or Amplitude · React · Statistics fundamentals	A/B test design doc · Results report · Rollout decision memo · PRD
	SQL Analytics — Direct Database Querying				
P26	Write SQL queries directly against a production-like database to answer business questions. Funnel queries, cohort retention, revenue by segment, churn analysis.	Data question formulation · SQL as PM tool · Analyst collaboration simulation · Dashboard design	Advanced SQL · Window functions · CTEs · Aggregations · Data visualization in Metabase or similar	PostgreSQL · Metabase or Redash · SQL	Query library · Dashboard spec · Data question log · PRD
	■ A PM who can write SQL is a PM who does not wait three days for an analyst to answer a question.				
	Customer Data Platform — Segment.io Integration				
P27	Integrate Segment.io as a central event router. Connect to downstream tools: analytics, email, CRM. Single event fires to all tools simultaneously.	CDP architecture understanding · Data governance basics · Single source of truth design	Segment.io · Source and destination configuration · Identity resolution · Event routing	Segment.io · Node.js · React · Mixpanel or Amplitude · SendGrid	CDP architecture diagram · Event routing map · PRD
	■ Closes Phase IV. Every growth-stage company uses a CDP. Knowing how it works makes you a better PM.				

Phase V · P28 – P33

REAL-TIME & SCALE — WebSockets, Queues, and Caching

Build products that operate at concurrency — multiple users, live updates, background processing, and cached responses. PM skill introduced: latency tolerance definition, concurrent user experience mapping, SLA writing, and infrastructure cost-benefit documentation.

Project	Scope	PM Growth	Tech Growth	New Tools	Artifacts
	Real-Time Chat Application with WebSockets				
P28	Multi-room chat with live message delivery, user presence indicators, typing status, and message history. Persistent via PostgreSQL.	Real-time UX spec · Presence design · Message delivery guarantee definition	WebSockets · Socket.io · Room-based event broadcasting · Presence tracking	Socket.io · Node.js · PostgreSQL · React	Real-time spec · Presence flow diagram · PRD
	Collaborative Live Document Editor				
P29	Multiple users editing a shared document simultaneously with live cursor positions, conflict-free merging, and operational status indicators.	Conflict resolution UX · Optimistic UI design · Collaborative feature scoping	Operational transforms or CRDT basics · Socket.io · Optimistic UI patterns · Redis pub/sub	Socket.io · Redis · Node.js · React	Concurrency model doc · Conflict resolution spec · PRD
	■ This is the hardest project in Phase V. The concurrency problem is genuinely complex.				
	Background Job Queue with BullMQ and Redis				
P30	Implement async background job processing: email delivery, report generation, image processing. Job retry logic, dead letter queue, job status dashboard.	Async vs sync decision framework · Job failure UX · SLA definition for background tasks	BullMQ · Redis · Job queue patterns · Retry/backoff · Worker processes	BullMQ · Redis · Node.js · PostgreSQL	Job taxonomy · Failure handling spec · SLA document · PRD
	Redis Caching Layer — Performance Optimization				
P31	Add Redis caching to a high-read endpoint. Cache invalidation strategy. Cache-hit rate measurement. Before/after latency comparison documented.	Performance requirement writing · Cache strategy documentation · Latency SLA definition	Redis caching · TTL strategy · Cache invalidation patterns · Google Lighthouse performance audit	Redis · Node.js · Google Lighthouse · Postman	Performance baseline report · Cache strategy doc · PRD
	Full-Text Search with Elasticsearch or Algolia				
P32	Add search to a content-heavy product. Indexed documents, typo tolerance, faceted filtering, relevance ranking, and search analytics.	Search relevance definition · Zero-results handling · Search analytics interpretation	Elasticsearch or Algolia · Index schema · Relevance tuning · Faceted filtering	Algolia or Elasticsearch · Node.js · React	Search spec · Relevance criteria · Zero-results strategy · PRD
	Notification System — In-App + SMS + Push				

Project	Scope	PM Growth	Tech Growth	New Tools	Artifacts
P33	Multi-channel notification system. In-app notification center, SMS via Twilio, and browser push notifications. User preference controls. Notification analytics.	Notification taxonomy design · Opt-in/opt-out UX · Notification fatigue strategy	Twilio SMS API · Web Push API · Notification preference storage · BullMQ for async delivery	Twilio · Web Push API · BullMQ · Redis · Node.js · React	Notification taxonomy · Preference spec · PRD
	■ Closes Phase V. Notification strategy is one of the most user-trust-sensitive decisions a PM makes.				

Phase VI · P34 – P41

AI & MACHINE LEARNING INTEGRATION

Integrate AI as a product capability — not as a novelty, but as a deliberate solution to a defined user problem. LLM integration, vector search, embeddings, RAG pipelines, and agentic workflows. PM skill introduced: AI product scoping, prompt engineering as product design, AI failure mode documentation, and responsible AI requirement writing.

Project	Scope	PM Growth	Tech Growth	New Tools	Artifacts
	OpenAI API Integration — Text Generation Feature				
P34	Add AI-powered text generation to an existing product. Prompt design, streaming responses, token usage tracking, error handling, and cost monitoring.	AI feature scoping · Prompt as product design · Cost-per-feature analysis · AI failure mode documentation	OpenAI API · Streaming responses · Token budgeting · Rate limiting · Error classification	OpenAI API or Anthropic SDK · Node.js · React · TypeScript	AI feature spec · Prompt library · Cost model · PRD
	Anthropic Claude API — Structured Output Product				
P35	Build a product feature using Claude that produces structured JSON output for downstream consumption. Multi-turn conversation state. System prompt engineering.	Structured output spec writing · System prompt as product requirement · Evaluation criteria for AI output	Anthropic SDK · System prompts · Multi-turn conversation management · JSON mode · Output validation	Anthropic SDK · Node.js · React · TypeScript · Zod (schema validation)	System prompt spec · Output schema · Evaluation rubric · PRD
	Vector Database and Semantic Search				
P36	Embed a document corpus and build semantic search over it. Similarity scoring, hybrid search (keyword + semantic), and relevance evaluation.	Semantic vs keyword search tradeoffs · Relevance evaluation design · Embedding cost analysis	Pinecone or Weaviate · OpenAI embeddings · Cosine similarity · Hybrid search · Chunking strategy	Pinecone or Weaviate · OpenAI Embeddings API · Node.js · TypeScript	Search architecture diagram · Relevance evaluation doc · PRD
	RAG Pipeline — Retrieval-Augmented Generation				
P37	Build a RAG system over a document corpus. Ingest, chunk, embed, retrieve, and generate. Document Q&A; with source citations. Hallucination mitigation documented.	RAG architecture documentation · Hallucination risk spec · Citation UX design · Grounding strategy	LangChain or custom RAG · Vector retrieval · Context window management · Source citation	LangChain · Pinecone · OpenAI or Anthropic · Node.js · TypeScript	RAG architecture diagram · Hallucination mitigation spec · PRD
	AI-Powered Document Analysis Tool				
P38	Upload a PDF or document. AI extracts structured data, generates summaries, answers questions, and flags key clauses. Multi-document comparison.	Document processing UX · Extraction accuracy definition · Edge case documentation	PDF text extraction · Multimodal inputs · Structured extraction prompts · Confidence scoring	Anthropic or OpenAI API · pypdf · Node.js · React · TypeScript	Extraction spec · Accuracy evaluation rubric · PRD

Project	Scope	PM Growth	Tech Growth	New Tools	Artifacts
	AI Chatbot with Memory and Persona				
P39	Persistent AI assistant with conversation memory, defined persona, tool use (web search or calculator), and guardrails. User feedback loop for quality improvement.	Persona specification · Guardrail requirement writing · Feedback loop design · Trust and safety spec	Conversation memory patterns · Tool use / function calling · Content filtering · Feedback storage	Anthropic or OpenAI API · LangChain or custom · PostgreSQL · React	Persona spec · Guardrail spec · Feedback loop design · PRD
	Machine Learning — Predictive Feature (Churn or Recommendation)				
P40	Train and deploy a simple ML model as a product feature. Churn prediction or content recommendation. Feature engineering, model evaluation, and A/B test against baseline.	ML feature requirements · Model evaluation criteria · False positive/negative cost analysis · Explainability requirement	Python · scikit-learn · Feature engineering · Model evaluation metrics · Model serving via API	Python · scikit-learn · pandas · FastAPI · PostgreSQL	ML feature spec · Model card · A/B test design · PRD
	■ First Python project. PM must understand model tradeoffs to write good ML requirements.				
	AI Agent with Tool Orchestration				
P41	Build an autonomous agent that can plan, use tools (search, code execution, API calls), and complete multi-step tasks with minimal human input. Observability built in.	Agentic product design · Human-in-the-loop spec · Observability requirements · Failure recovery design	LangChain agents or custom · Tool definitions · Reasoning loop patterns · Tracing and observability	Anthropic or OpenAI API · LangChain · Python or Node.js · LangSmith or similar	Agent architecture diagram · Tool inventory · Observability spec · PRD
	■ Closes Phase VI. Agentic AI is the current frontier. This project is deliberately placed after deep foundations.				

Phase VII · P42 – P50+

PLATFORM, MOBILE & PRODUCTION SCALE

Graduate from solo product builder to platform thinker. GraphQL, mobile-native development, micro-frontend architecture, Docker, CI/CD pipelines, and multi-tenant SaaS. PM skill introduced: platform strategy, API versioning, mobile product management, infrastructure cost modeling, and enterprise-grade technical documentation. Portfolio milestone projects emerge from this phase.

Project	Scope	PM Growth	Tech Growth	New Tools	Artifacts
	GraphQL API with Apollo Server				
P42	Replace or augment a REST API with GraphQL. Schema design, resolvers, mutations, subscriptions, and N+1 query optimization via DataLoader.	GraphQL vs REST decision · GraphQL documentation · Schema design as product contract · API versioning strategy	GraphQL · Apollo Server · Resolvers · DataLoader · Schema-first design	GraphQL · Apollo Server · Apollo Client · Node.js · TypeScript	API schema doc · REST vs GraphQL decision memo · PRD
	React Native Mobile App (Cross-Platform)				
P43	Port a web product to iOS and Android using React Native. Navigation, device APIs, push notifications, offline mode, and app store submission prep.	Mobile-specific UX requirements · Platform parity decisions · App store submission spec · Offline-first design	React Native · Expo · React Navigation · Device APIs · Push notifications (APNs/FCM)	React Native · Expo · TypeScript · React Navigation	Mobile PRD · Platform parity matrix · App store submission checklist
	Containerization with Docker				
P44	Containerize a full-stack application. Multi-container setup with Docker Compose (app, database, Redis, worker). Production-ready Dockerfile patterns.	Infrastructure requirement writing · Environment parity documentation · Deployment risk documentation	Docker · Docker Compose · Multi-stage builds · Environment variable management	Docker · Docker Compose · Node.js · PostgreSQL · Redis	Deployment architecture diagram · Environment spec · PRD
	CI/CD Pipeline with GitHub Actions				
P45	Full CI/CD pipeline: automated tests on PR, build validation, staging deploy, manual production gate, and rollback procedure. Zero-downtime deploy confirmed.	Release process documentation · Deploy gate criteria · Rollback decision tree · Change management spec	GitHub Actions · Workflow YAML · Test automation in CI · Environment secrets management	GitHub Actions · Docker · Render or Railway or AWS	CI/CD pipeline diagram · Deploy runbook · Rollback procedure · PRD
	Multi-Tenant SaaS Architecture				
P46	Extend an existing product to support multiple organizations (tenants). Data isolation strategy, tenant-scoped API, org-level admin, and billing per tenant.	Multi-tenant scoping · Data isolation requirement writing · Tenant onboarding flow · Per-tenant pricing model	Row-level security · Tenant context middleware · Schema-per-tenant vs shared schema tradeoffs	PostgreSQL (RLS) · Node.js · Stripe · React	Tenant architecture diagram · Data isolation spec · Onboarding flow · PRD
	■ Multi-tenancy is the architectural foundation of every B2B SaaS product.				

Project	Scope	PM Growth	Tech Growth	New Tools	Artifacts
	Internationalization and Localization (i18n/l10n)				
P47	Add multi-language support to a product. String externalization, locale switching, RTL layout support, date/number/currency formatting per locale.	Localization requirement writing · Market entry documentation · Translation workflow design	i18next · React-i18next · RTL CSS · Locale-aware formatting · Translation file management	i18next · React · Node.js · Crowdin or similar (optional)	Localization spec · Locale support matrix · Translation workflow · PRD
	Micro-Frontend Architecture				
P48	Decompose a large frontend into independently deployable micro-frontends. Module Federation, shared design system, and independent team deployment simulation.	Platform architecture documentation · Team topology design · API boundary definition	Webpack Module Federation · Shared component library · Independent deployment pipelines	Webpack Module Federation · React · TypeScript · GitHub Actions	Platform architecture diagram · Team boundary spec · PRD
	Observability Stack — Logging, Metrics, Alerts				
P49	Instrument a production application with structured logging, performance metrics, error tracking, uptime monitoring, and on-call alerting.	SLA and SLO definition · Incident response documentation · On-call runbook writing	Structured logging · Sentry (error tracking) · Uptime monitoring · Alert threshold design	Sentry · Datadog or Grafana · PagerDuty or similar · Node.js	SLA/SLO doc · Alert threshold spec · Incident runbook · PRD
	Portfolio Milestone Projects — TBD at Threshold				
P50+	Determined by coach assessment at skill threshold. Projects in this range are the first portfolio-worthy products — evaluator-relevant, fully instrumented, and built to production standard across all nine PMD phases.	Full PM lifecycle ownership · Cross-functional artifact suite · Business case development · GTM strategy	Full stack synthesis · All prior capabilities integrated · Production deployment · Monitoring live	Full stack — determined per project at scoping	Complete PMD artifact suite · Case study · Portfolio showcase entry
	■ <i>This threshold is not self-declared. The coach assesses readiness based on the quality of P40–P49 output and retrospective depth. The portfolio milestone is earned, not scheduled.</i>				

RECOMMENDED READING — BY PHASE

I–II · Foundation	• Don't Make Me Think — Steve Krug · UX fundamentals every PM must internalize • The Pragmatic Programmer — Hunt & Thomas · Developer mindset for non-developers • HTML & CSS: Design and Build Websites — Jon Duckett · Visual learner's entry point
II–III · Framework & Backend	• Clean Code — Robert C. Martin · Writing code your future self won't curse • Designing Data-Intensive Applications — Martin Kleppmann · The PM's database bible • The Phoenix Project — Kim, Behr, Spafford · Engineering culture through fiction
III–IV · Analytics & Growth	• Lean Analytics — Croll & Yoskovitz · AARRR, cohorts, and the one metric that matters • Hacking Growth — Ellis & Brown · Systematic growth experimentation • Trustworthy Online Controlled Experiments — Kohavi et al. · A/B testing rigorously
IV–V · Product Strategy	• Inspired — Marty Cagan · How the best product teams work • Continuous Discovery Habits — Teresa Torres · Opportunity trees and user interviews • Competing Against Luck — Clayton Christensen · Jobs-to-be-Done theory at depth
V–VI · AI & Platform	• Building Machine Learning Powered Applications — Emmanuel Ameisen • AI Product Management — Theresa Yang · Practical AI scoping and ethics • Platform Revolution — Parker, Van Alstyne, Choudary · Network effects and platform strategy
VI–VII · Leadership & Scale	• The Lean Startup — Eric Ries · Build-measure-learn at every scale • Measure What Matters — John Doerr · OKRs in practice • Obviously Awesome — April Dunford · Positioning as a PM discipline

PMD · Artifact #18 · Project Roadmap Architecture v1.0 · May 2026 · Where brilliant ideas become shipped products